

Architektura systému

Třívrstvá architektura

Aplikace využívá standardizovanou třívrstvou architekturu, která umožňuje logické oddělení uživatelského rozhraní, byznys logiky a datové persistence.

1. **Prezenční vrstva:** Zodpovídá za interakci s uživatelem. Využívá šablonovací systém **Thymeleaf**, který transformuje data z backendu do finální HTML podoby. Tato vrstva neobsahuje žádnou byznys logiku, pouze data zobrazuje nebo sbírá vstupy z formulářů.
2. **Aplikační vrstva:** Představuje „mozek“ celého systému. Zde probíhají veškeré výpočty (např. dynamický výpočet pořadí – Rank), validace (např. kontrola duplicitních registrací) a orchestrace datových toků. Tato vrstva komunikuje směrem nahoru s kontrolery a směrem dolů s repozitáři.
3. **Datová vrstva:** Zajišťuje trvalé ukládání dat. Je realizována pomocí **Spring Data JPA** a jako fyzické úložiště využívá databázový stroj **SQLite**. Díky abstrakci pomocí Hibernate je tato vrstva nezávislá na konkrétním SQL dialektu.

Využití návrhového vzoru MVC

V rámci frameworku Spring Boot aplikace striktně implementuje vzor MVC, který definuje tok dat v aplikaci:

- **Model:** Představuje datové entity (User, Event, Category, Registration) a DTO objekty, které nesou informace mezi vrstvami.
- **View:** Představuje dynamické HTML šablony (Thymeleaf), které jsou interpretovány prohlížečem uživatele.
- **Controller:** Funguje jako prostředník. Přijímá HTTP požadavky od uživatele, volá příslušné služby v aplikační vrstvě a následně vybírá odpovídající View, kterému předá zpracovaná data.

Princip Server-Side Rendering (SSR)

O:Pocket využívá přístup **Server-Side Rendering**. Při každém požadavku (např. zobrazení detailu závodu) server provede následující kroky:

1. Kontroler přijme požadavek a vyžádá si data ze služby.
2. Služba načte surová data z databáze a provede nezbytné výpočty (např. seřadí běžce podle času).
3. Server vezme čistou HTML šablonu a „vstříkne“ do ní tato data.
4. Uživateli je odeslán již kompletně sestavený HTML kód.

Datový model

Tato kapitola popisuje strukturu uložených dat a vztahy mezi jednotlivými entitami v databázi SQLite.

Entity a atributy

User (Uživatel)

- `id` (PK) – integer
- `email` – varchar(255)
- `first_name` – varchar(255)
- `last_name` – varchar(255)
- `password` – varchar(255)
- `registration_number` – varchar(255)
- `username` – varchar(255)

Event (Závod)

- `id` (PK) – integer
- `name` – varchar(255)
- `date` – date
- `location` – varchar(255)

- map_name – varchar(255)
- organizer_id (FK) – bigint

Category (Kategorie)

- id (PK) – integer
- event_id (FK) – bigint
- name – varchar(255)
- length – float
- climbing – integer

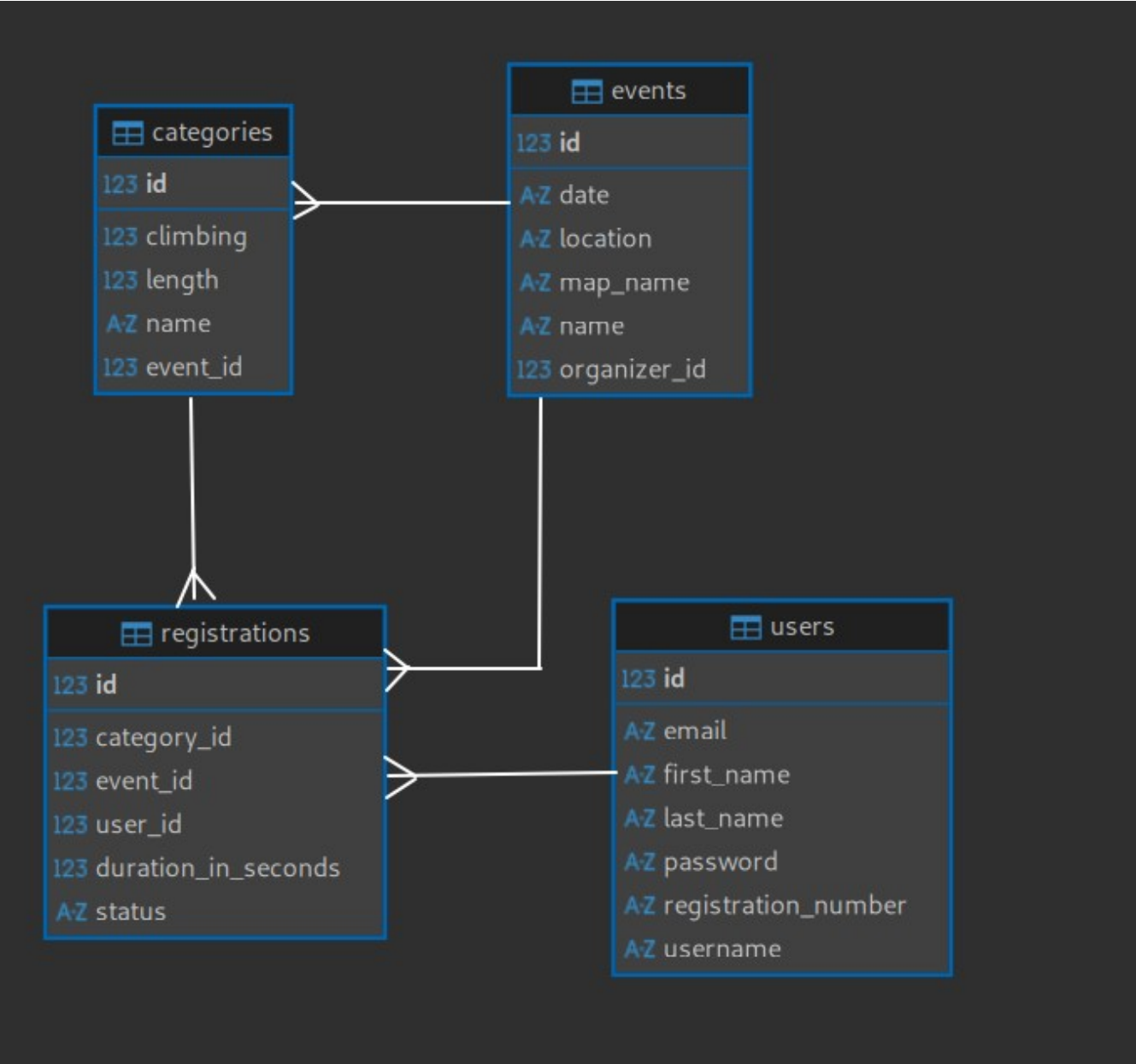
Registration (Registrace)

- id (PK) – integer
- user_id (FK) – bigint
- category_id (FK) – bigint
- event_id (FK) – bigint
- duration_in_seconds – integer
- status – varchar(255)

Databázové vztahy (Relations)

- **Event 1:N Category:** Jeden závod může obsahovat libovolné množství kategorií. Smazání závodu vyvolá kaskádové smazání všech kategorií.
- **Category 1:N Registration:** Do jedné kategorie se může přihlásit mnoho běžců.
- **User 1:N Registration:** Jeden uživatel může mít v systému mnoho registrací (historie startů), ale v rámci jednoho závodu je omezen unikátním indexem na jednu aktivní registraci.
- **Event 1:N Registration:** Přímá vazba umožňující rychlý přístup ke všem účastníkům akce bez nutnosti procházení kategorií.

ERD diagram



Návrh endpointů (URL Routing)

Tato kapitola definuje mapování URL adres na konkrétní akce v kontrolerech. Jelikož aplikace využívá **Server-Side Rendering**, každý endpoint vrací buď celou HTML stránku (přes šablonu Thymeleaf), nebo provádí přesměrování (Redirect) po úspěšném zpracování formuláře.

Uživatelské rozhraní a autentizace

Metoda	Endpoint	Popis akce
GET	/	Zobrazení přihlašovací stránky (Login).
POST	/login	Zpracování přihlášení uživatele.
GET	/register	Zobrazení registračního formuláře.
POST	/register	Vytvoření nového uživatelského účtu v tabulce users.
GET	/logout	Ukončení uživatelské relace.
GET	/feed	Hlavní přehled všech vypsanych závodů.

Správa profilu a historie

Metoda	Endpoint	Popis akce
GET	/profile	Osobní profil uživatele.
GET	/profile/edit	Formulář pro úpravu osobních údajů.
POST	/profile/edit	Uložení změn v profilu uživatele.
GET	/my-registrations	Seznam závodů, na které je uživatel aktuálně přihlášen.
GET	/race-history	Kompletní historie všech absolvovaných startů uživatele.

Management závodů a výsledků

Metoda	Endpoint	Popis akce
GET	/my-events	Seznam závodů, které aktuálně přihlášený uživatel organizuje.
GET	/event/add	Formulář pro vytvoření nového závodu.
POST	/event/add	Zápis nového závodu do tabulky events.
GET	/event/edit	Editace detailů již existujícího závodu.
POST	/event/edit	Uložení změn v parametrech závodu (např. map_name).
GET	/event/categories	Správa kategorií pro konkrétní závod.
POST	/event/categories/add	Přidání nové kategorie (vazba na event_id).
POST	/event/categories/delete	Odstranění kategorie ze závodu.
GET	/event/manage-results/{eventId}	Rozhraní pro vkládání výsledků u konkrétního závodu.
POST	/event/save-single-result	Uložení nebo aktualizace času a statusu pro jednoho běžce.

Účast na závodě

Metoda	Endpoint	Popis akce
GET	/event/join	Formulář pro výběr kategorie a přihlášení na závod.
POST	/event/join	Vytvoření záznamu v tabulce registrations.
POST	/event/unregister	Odhlášení uživatele ze závodu (smazání záznamu).
GET	/event/results/{eventId}	Veřejné zobrazení výsledkové listiny daného závodu.

Návrh byznys logiky

Dynamický výpočet pořadí (Rank)

Jednou z nejdůležitějších funkcí je výpočet umístění závodníka v rámci kategorie. Hodnota **Rank** není v databázi uložena staticky, aby se předešlo nekonzistenci při dodatečných úpravách výsledků (např. při diskvalifikaci jiného závodníka).

Výpočet probíhá v reálném čase podle následujícího algoritmu:

1. Systém vyfiltruje všechny registrace, které patří do stejné kategorie.
2. Z výběru jsou uvažováni pouze závodníci se statusem OK.

Tento přístup zajišťuje, že výsledková listina je vždy aktuální a reflektuje veškeré změny v datech bez nutnosti manuálního přepočtu.

Validace registračního procesu

Aby byla zajištěna integrita sportovní soutěže, implementuje RegistrationService kontrolní mechanismy při každém pokusu o přihlášení (endpoint /event/join):

- **Kontrola duplicity:** Před zápisem do tabulky registrations systém ověří, zda uživatel již nemá v rámci stejného event_id existující záznam.
- **Konzistence kategorií:** Systém ověřuje, zda zvolená kategorie skutečně náleží k danému závodu, čímž brání neautorizované manipulaci s URL parametry.

Převod a validace času

Organizátor zadává výsledky v přehledném formátu, ale systém vnitřně pracuje s jednotkou **sekundy** (atribut duration_in_seconds).

- Při uložení probíhá konverze z uživatelského vstupu na celočíselnou hodnotu.
- Tato standardizace umožňuje efektivní využití matematických operátorů přímo v databázových dotazech (SQL ORDER BY) a zrychluje generování výsledkových listin pro velké počty účastníků.

Technologický stack

Tato kapitola shrnuje veškeré technologie, frameworky a vývojové nástroje použité při realizaci systému **O:Pocket**.

Přehled použitých technologií

Vrstva / Nástroj	Technologie	Poznámka
Backend framework	Spring Boot 3.x	Zajišťuje orchestraci celé aplikace a dependency injection.
Programovací jazyk	Java 17	Využití moderních prvků jazyka (Records, Sealed Classes).
Šablonovací systém	Thymeleaf 3	Zpracování HTML na straně serveru (SSR).
Persistence (ORM)	Spring Data JPA	Abstrakce nad databází pomocí Hibernate.
Databázový stroj	SQLite	Lokální relační databáze s nulovou konfigurací.
Zabezpečení	Spring Security	Autentizace a autorizace uživatelských rolí.
Build nástroj	Maven	Správa závislostí a automatizace buildu.
Verzování kódu	Git / GitHub	Správa zdrojových kódů a historie změn.

Vývojové prostředí a nástroje

- **IDE:** IntelliJ IDEA – primární nástroj pro psaní kódu a ladění aplikace.
- **Database Tool:** DB Browser for SQLite + Terminál – pro manuální kontrolu dat a ověřování SQL dotazů.
- **OS:** Ubuntu Linux